



**សាកលវិទ្យាល័យភូមិន្ទភ្នំពេញ**

Royal University of Phnom Penh

**Maintenance / Support Documentation**



# 1. Purpose

This section explains how the system is maintained and supported after deployment in a **microservice-based architecture**. It focuses on future updates, independent service deployment, and issue resolution.

## 1.1. Maintenance Guide

The system is divided into multiple independent microservices. Each microservice can be updated, debugged, and deployed separately without affecting the entire system. In Local Advertising System there are 3 main services that we used such as User Service, Ads Service, Billing Service.

### 1.1.1. Service Update & Deployment

In this system, each microservice is containerized using Docker. When an update is required, only the affected service container is updated and redeployed.

The update process starts by modifying the source code of the selected microservice. After the code changes, a new Docker image is built for that service. The updated container is then started, replacing the previous version, while other service containers continue running.

Basic verification is performed after deployment to ensure the container runs successfully and the service responds correctly to requests. This verification confirms that the updated service is functioning as expected.

Make sure use docker compose build ensure that it doesn't contain any errors here before push into deployment.

### 1.1.2. Debugging a Microservices

This section explains how to identify and fix issues in a microservice independently, without affecting other services.

#### Steps for Debugging a Microservice

- Identify the Affected Service :  
Determine which microservice is experiencing the issue.  
*Example:* Users cannot log in → affected service: **Authentication Service**

- **Check Service Logs :**  
Examine the logs of the affected microservice to find error messages.  
*Example:* “Invalid token generation” or “Database connection error” appears in logs.
- **Reproduce the Issue :**  
Reproduce the problem in a development or test environment.  
*Example:* Test login locally to observe the token error.
- **Fix the Issue :**  
Correct the code or configuration causing the problem.  
*Example:* Fix the login validation logic in Authentication Service.
- **Test the Service :**  
Test only the affected microservice independently to ensure the fix works.  
*Example:* Call the login API endpoint and confirm successful login.
- **Redeploy the Service Using Docker :**  
Build a new Docker image for the updated service and start the container.  
Other services remain running.  
*Example:* `docker build -t auth-service:v1.1 .` then `docker run -d --name auth-service auth-service:v1.1` or `docker compose run build`
- **Verify Service Communication :**  
Ensure the updated microservice interacts properly with other services.  
*Example:* Authentication Service works correctly with User and Order Services.
- **Update Documentation / Release Notes:**  
Record the fixed issue and any important notes.  
*Example:* “Fixed login validation bug in Authentication Service, version v1.1.0”

Each microservice can be debugged independently. The logs are checked, issues reproduced and fixed, the service is tested and redeployed using Docker, and documentation is updated accordingly.

### 1.1.3. System / Service Enhancement

This section defines the procedure for improving or extending a microservice after deployment. Enhancements may include performance optimization, feature addition, or code refactoring, while ensuring minimal impact on other services.

#### Enhancement Workflow

- Requirement Identification
  - Gather enhancement requests or identify performance bottlenecks.
  - Assess impact on dependent microservices and shared resources.
- Impact Analysis
  - Evaluate potential effects on system stability, APIs, and data.
  - Document risks and mitigation strategies.
- Code Implementation
  - Update the microservice codebase in accordance with requirements.
  - Ensure backward compatibility and adherence to coding standards.
- Testing
  - Conduct unit, integration, and functional tests targeting only the updated service.
  - Verify correct interaction with dependent services via API calls.
- Containerization and Deployment
  - Build a new Docker image for the updated microservice.
  - Redeploy the service container with zero downtime or minimal disruption to other services
- Post-Deployment Verification
  - Monitor logs, service health, and API responses.
  - Confirm seamless interaction with other microservices.
- Documentation and Release Notes
  - Update internal documentation with enhancement details.
  - Record changes, version number, and deployment notes in release logs.

Service enhancements are performed independently per microservice, using a controlled workflow to ensure system stability, maintainability, and traceable documentation.

